Does a Failure Monitor Survive Compression of the Policy It Watches?

Anonymous Author(s)
Affiliation
Address
email

Abstract:

Keywords: VLA, quantization, runtime monitoring, failure detection, conformal prediction

1 Introduction

A vision-language-action (VLA) policy that reaches a robot is usually changed twice before deployment. It is compressed, because a 7B or even 0.5B model is slow at control rates [1, 2]. And it is wrapped in a runtime failure monitor, increasingly one that reads the policy's own hidden states and raises an alarm when a calibrated threshold is crossed [3, 4, 5]. The monitor is trained and calibrated on rollouts of the policy as it was trained. Nobody, as far as we can find, checks whether it still works on the policy as it is deployed.

Compression papers report that weight quantization leaves task success almost unchanged [1, 6]. That is the wrong evidence for a monitor. A hidden-state monitor depends on the geometry of the representation, and a conformal threshold depends on calibration and deployment rollouts being exchangeable; neither is implied by equal success rates. In language models, quantization that improves perplexity still damages a fifth of an SAE's features [7].

We test this directly.

2 Setup

Policy and task. We use SmolVLA [8] (the public LIBERO checkpoint, $\sim$450M parameters, SmolVLM2 backbone with a flow-matching action expert) through LeRobot's evaluation stack, with the checkpoint's own settings: 50-step action chunks, replanning every step, 10 Euler steps. We evaluate on LIBERO-Long [9] (10 tasks, 50 fixed initial states each, up to 520 steps), the suite where the policy fails most often. Our rollout driver reproduces LeRobot's `lerobot-eval` on matched tasks and initial states.

Compression. Four conditions: **ref**, the checkpoint as shipped; **int8 W**, round-to-nearest weight-only int8 with per-channel scales; **int4 W**, weight-only int4 with groups of 128 and zero points; and **2 flow steps**, Euler integration with 2 steps instead of 10 and the weights untouched. The quantizers cover all 521 linear layers of the vision encoder, language model and action expert (508M parameters). The policy's small projection heads stay in full precision. Quantization is simulated (quantize, then dequantize), so it reproduces the numerics of an int8/int4 kernel but not its speed. We check that every quantized group holds at most 2^b distinct values, after a recent report found that "INT8" VLA artifacts were in fact FP32 [10]. The 2-step condition is a real speedup: it cuts policy time per call by $3.2\times$ on an L4.

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

Monitors. Four monitors, one from each family in recent work, are each fit once on reference rollouts and never refit:

- **probe (VLM state):** a logistic probe on the VLM's mean-pooled final-layer state, trained on per-step states labeled with the episode outcome (SAFE [3]);
- **probe (expert state):** the same probe on the action expert's state at the last denoising step;
- **Mahalanobis (VLM):** distance of the VLM state from successful training steps, using 64 principal components and a Ledoit–Wolf covariance. It needs no failure labels (VLA-FAIL [5], FIPER [11]);
- **action consistency:** disagreement between consecutive action chunks on their 49 shared steps. It needs no training (Sentinel [12], VLA-FAIL).

Each monitor gives a per-step score, and the episode score at step t is its running mean S_t. An alarm fires at the first $t \geq 10$ with $S_t > \tau$. The threshold τ is the conformal $(1-\alpha)$ quantile, with $\alpha = 0.1$, of $\max_{t \geq 10} S_t$ over successful *reference* calibration episodes [13]. If deployment episodes were exchangeable with the calibration episodes, a successful episode would raise a false alarm with probability at most α. Compression is exactly the change that breaks that assumption.

Splits and protocol. Splits are by initial state and are shared across tasks. States 0–29 are reference-only and train the monitors. States 30–39 are calibration. States 40–49 are the test set, and every number we report comes from them. That gives 500 reference episodes and 200 per compressed condition. Hypotheses, splits and analyses were pre-registered before data collection; we mark anything else as exploratory.

3 Results

4 Related work

VLA failure monitors. SAFE [3] probes VLA hidden states and calibrates alarms with functional conformal prediction. SAFECAST [4] shows that such thresholds go wrong under deployment-time *input* shift (clutter, lighting, instructions) and adds contrast sets to calibration. VLA-FAIL [5] pairs a last-layer Mahalanobis detector with action-chunk consistency. FIPER [11] combines embedding novelty with action-chunk entropy, and Sentinel [12] uses temporal consistency together with a VLM progress check. RESCUE [14] argues that monitors should be judged by the interventions they trigger. All of these evaluate a monitor on the policy it was fit to. We hold the inputs fixed and change the policy underneath the monitor.

VLA compression. Post-training quantization of VLAs has been benchmarked at scale [1], alongside new quantizers [2] and studies of what compression does to motion quality [6]. These studies measure success, latency and trajectory metrics; none examines the monitors deployed alongside the policy. In language models, Duan [7] shows that quantization can damage SAE features while perplexity is unchanged. We ask the analogous question for the monitors a robot actually acts on.

5 Discussion and limitations

References

[1] J. Xu, Q. Jin, M. Chen, S. Wang, Y. Sui, and Y. Shi. VLAQuantBench: Closed-loop evaluation of post-training quantization for vision-language-action models. *arXiv preprint arXiv:2609.25376*, 2026.

[2] J. Zhang, Y. Hsieh, Z. Wan, H. Lin, X. Wang, Z. Wang, et al. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026.

[3] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, et al. SAFE: Multitask failure detection for vision-language-action models. *arXiv preprint arXiv:2506.09937*, 2025.

[4] H. Rajaprakash, A. Prajapati, R. Xue, A. Anwar, and J. Thomason. SAFECAST: Robust failure detection for VLA policies with contrast-set training and calibration. *arXiv preprint arXiv:2608.04246*, 2026.

[5] F. Seligmann, E. Gospodinov, E. U. Dincer, and G. Neumann. VLA-FAIL: Efficient task failure detection for finetuned vision-language-action models. *arXiv preprint arXiv:2606.21386*, 2026.

[6] Z. Li, H. Yang, Z. Chen, Y. Chen, et al. From inference efficiency to embodied efficiency: Revisiting efficiency metrics for vision-language-action models. *arXiv preprint arXiv:2603.19131*, 2026.

[7] E. Duan. Perplexity can miss SAE feature damage under quantization. *arXiv preprint arXiv:2606.03002*, 2026.

[8] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2023.

[10] R. Islam. When faster VLA deployment changes closed-loop behavior: Task success-latency analysis of SmolVLA across PyTorch and ONNX variants. *arXiv preprint arXiv:2609.14146*, 2026.

[11] R. Römer, A. Kobras, L. Worbis, and A. P. Schoellig. Failure prediction at runtime for generative robot policies. *arXiv preprint arXiv:2510.09459*, 2025.

[12] C. Agia, R. Sinha, J. Yang, Z.-a. Cao, R. Antonova, M. Pavone, and J. Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning*, 2024.

[13] A. N. Angelopoulos and S. Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[14] D. Ahmedt-Aristizabal, A. Khamis, M. A. Armin, and L. Petersson. RESCUE: Is this alarm worth acting on? counterfactual evaluation of VLA runtime monitors. *arXiv preprint arXiv:2610.13516*, 2026.